Assessment Details	Please Complete All Details
Group	A
Module Title	IY4113 Applied Software Engineering using Object-Orientated Programming
Assessment Type	Assigment
Module Tutor Name	Jonathan Shore
Student ID Number	T0466145
Date of Submission	29/04/2026
Word Count	3531

- [x] *I confirm that this assignment is my own work. Where I have referred to academic sources, I have provided in-text citations and included the sources in the final reference list.*

- [x] *Where I have used AI, I have cited and referenced appropriately.

Research (minimum of 2, at least 3)

Conduct research to support your coding process, including use of code examples, tutortials, documentation and AI tools (if used). Use the structure below to capture your evidence:

---

*Title of research: *Object Oriented Programming in java *Reference (link):J*ava OOP(Object Oriented Programming) Concepts - GeeksforGeeks *How does the research help with coding practise?:*

This research helped me better understand how object-oriented programming can organize a Java app into separate classes, each with its own job (GeeksforGeeks, 2024). The article explained key ideas like encapsulation, abstraction, inheritance, and polymorphism. It showed me why big programs should be split into smaller, reusable classes instead of putting everything

into one file or method. This makes the code easier to read, maintain, debug, and grow over time.

For the CityRide Lite project, this research led to creating separate classes like:

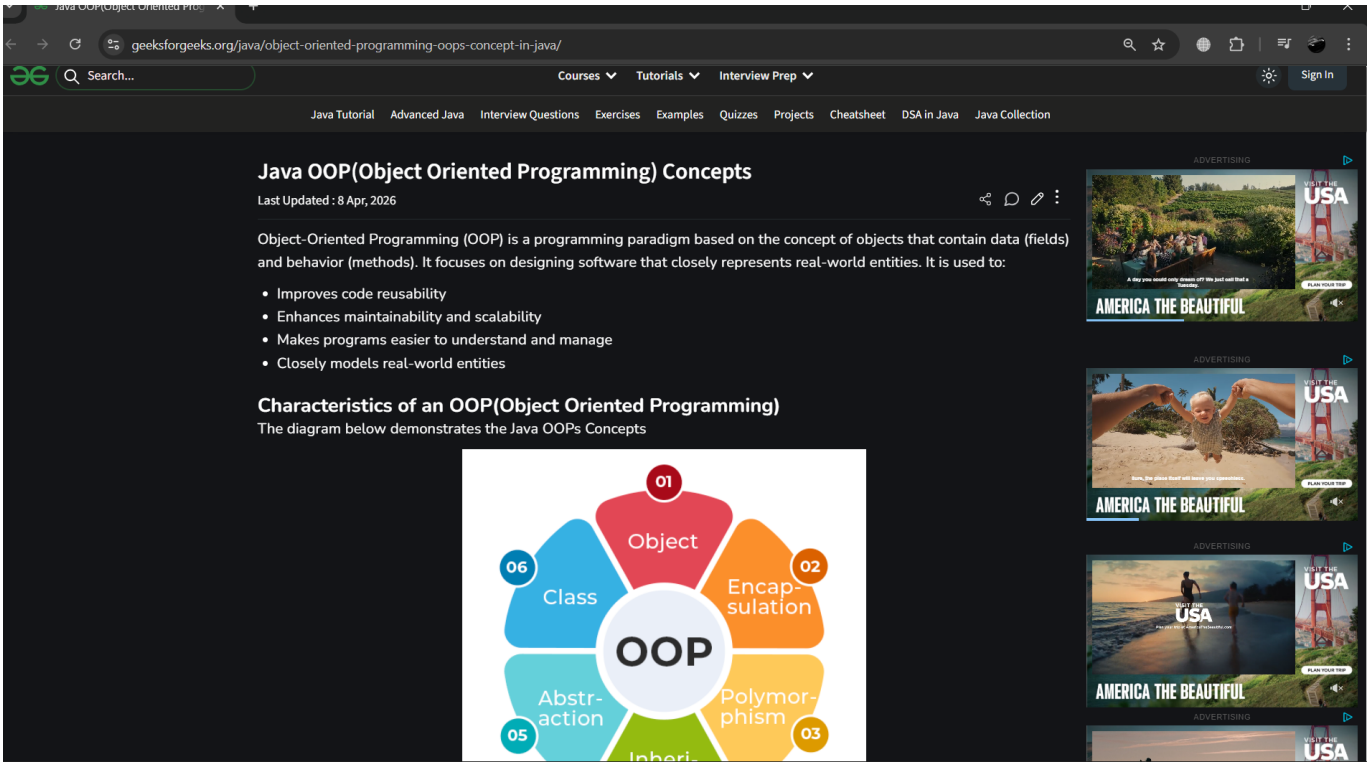
- Journey
- FareCalculator
- JourneyManager
- InputValidator
- Menu

This setup follows good coding practices and makes the system easier to manage and expand (GeeksforGeeks, 2024).

Key coding ideas you could reuse in your program:

- Put different jobs in different classes
- Write methods you can use more than once
- Use private variables and getters to access them
- Break big tasks into smaller, simpler steps
- Keep methods short and easy to read
- Split code into separate parts so it's easier to fix and update

Screenshot of research:



Title of research:Java Collections Reference (link):Lesson: Custom Collection Implementations (The Java™ Tutorials > Collections) How does the research help with coding practise?:

The Oracle Collections Framework Documentation explains that HashMap is useful for storing data as key-value pairs (Oracle, 2025). It allows you to quickly find, add, or remove items using a unique key.Using a HashMap encourages good coding practices in several ways. First it makes the code more efficient because finding an item by its key is very fast, even when there are many items. Second it keeps the code clean and simple since you don't need to write extra loops to search for a specific journey. Third it reduces errors because each journey has a unique key, so you won't accidentally mix up different journeys. Finally it makes the code easier to read and understand because the key clearly identifies what each journey represents for example using a journey ID as the key.

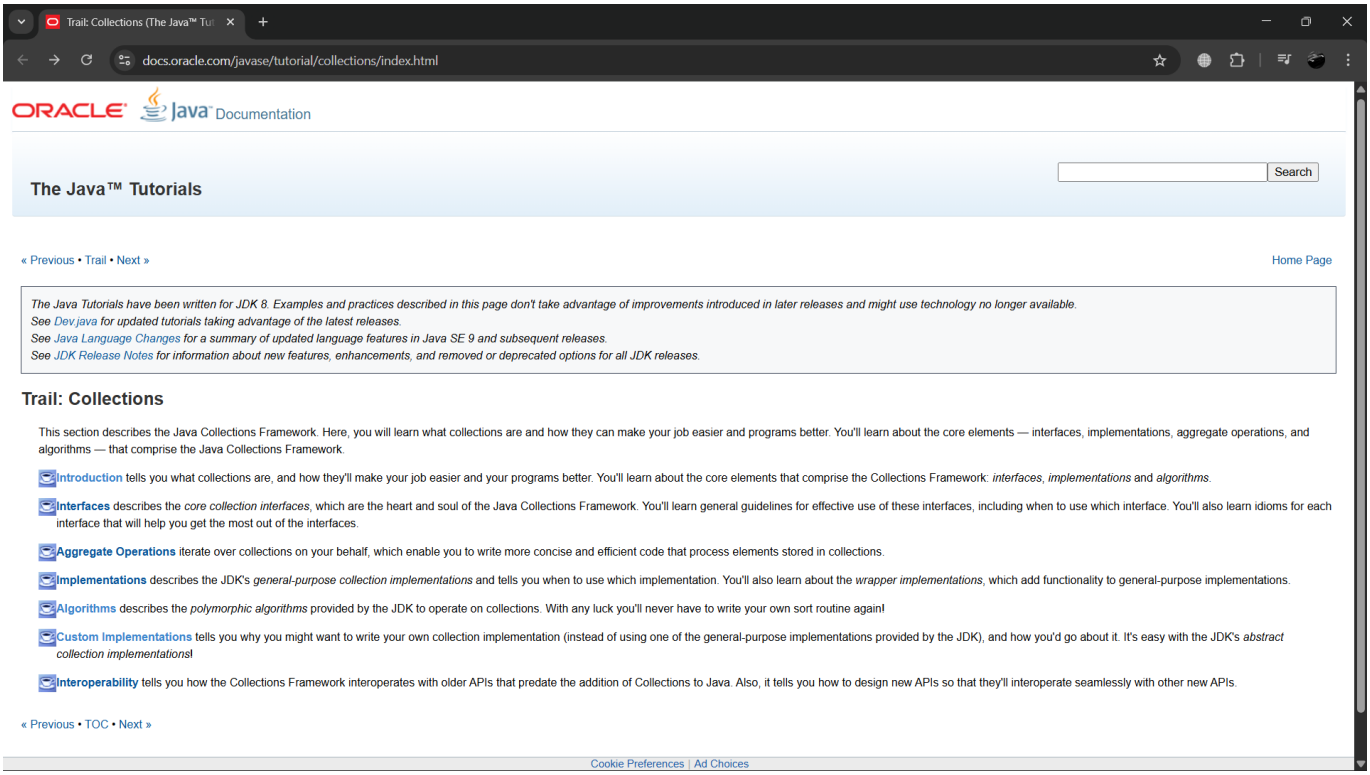
This influenced the decision to store all journey objects inside aHashMap<String, Journey> within the JourneyManager class.

Key coding ideas you could reuse in your program:

- Added using a unique ID (like a journey number)
- Retrieved quickly by that ID
- Removed easily

- Checked for existence efficiently (Oracle, 2025)

Screenshot of research:



Title of research: Input Validation

reference(link): Java - Exceptions

According to TutorialsPoint Java Exception Handling, validation and exception handling make software more reliable and reduce errors while the program is running (TutorialsPoint, 2024).

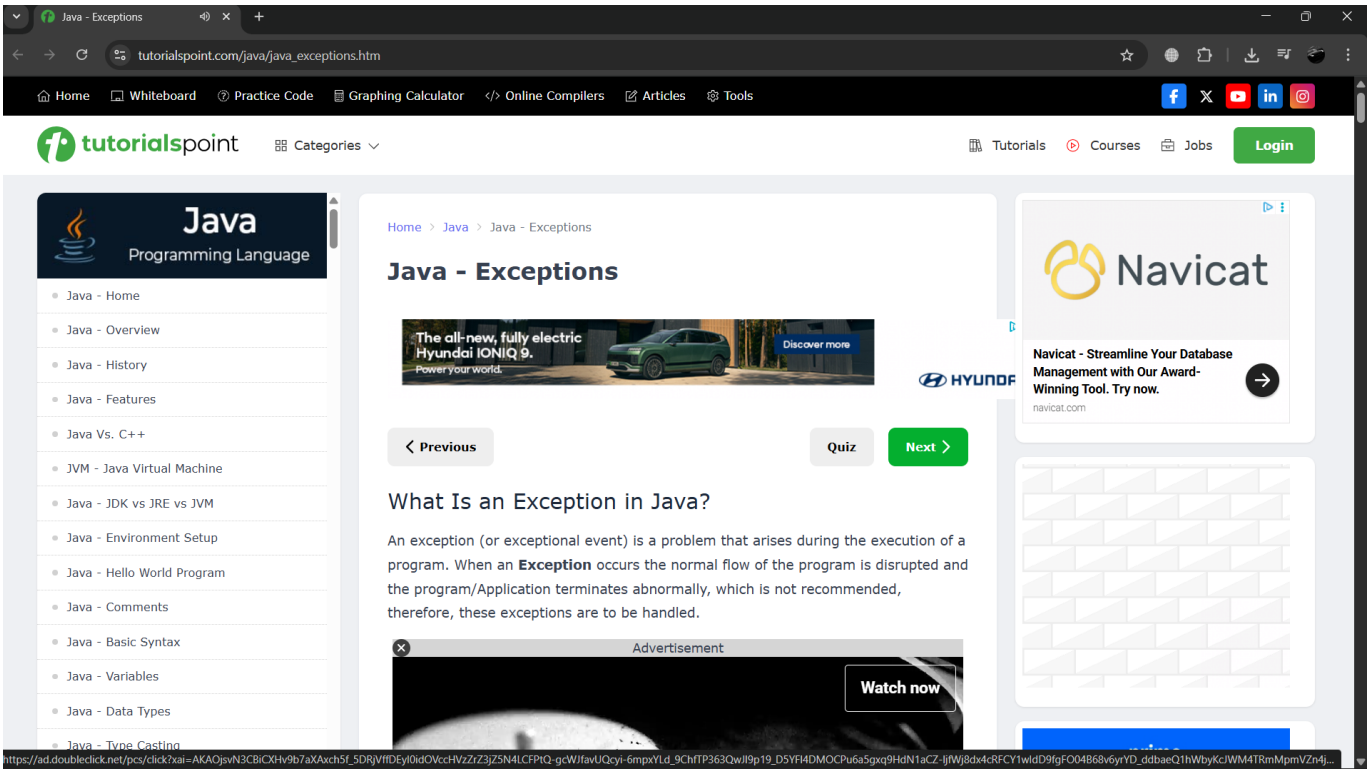
How this helps with coding practices:

Using input validation improves coding practices in several ways. First it stops the program from crashing when a user enters something invalid, like a letter instead of a number. Second it makes the user experience better because the program gives clear error messages and lets the user try again instead of just failing. It keeps the program's data clean and correct which prevents bugs later on. It makes the code easier to debug because errors are caught early at the point of input rather than deep inside the program logic. Finally it follows professional coding standards where a program should never trust user input without checking it first.

This research influenced the following types of validation in the CityRide Lite project:

- Zone validation (checking if the zone number is between 1 and 5)
- Menu validation (checking if the user picks a valid menu option)

- Passenger type validation (checking if the passenger type is recognised)
- Time band validation (checking if the time band is valid)



Program Code

Paste the current program code created so far. It does not have to be runnable code (document though if it does not work!)

Program code goes here: **Filename: Journey.java**

```
import java.math.BigDecimal;

import java.math.RoundingMode;

import java.time.LocalDate;

import java.time.format.DateTimeFormatter;

public class Journey {
```

```
// Private fields
private int journeyId;
private LocalDate date;
private CityRideDataset.TimeBand timeBand;
private int fromZone;
private int toZone;
private CityRideDataset.PassengerType passengerType;
private int zonesCrossed;
private BigDecimal baseFare;
private BigDecimal discountApplied;
private BigDecimal chargedFare;


 // Constructor for creating a new journey.

public Journey(int journeyId, LocalDate date, CityRideDataset.TimeBand timeBand,
               int fromZone, int toZone, CityRideDataset.PassengerType passengerType) {
    this.journeyId = journeyId;
    this.date = date;
    this.timeBand = timeBand;
    this.fromZone = fromZone;
    this.toZone = toZone;
    this.passengerType = passengerType;
    this.zonesCrossed = 0;
    this.baseFare = BigDecimal.ZERO;
    this.discountApplied = BigDecimal.ZERO;
    this.chargedFare = BigDecimal.ZERO;
}


 // Calculates zones crossed using formula: |toZone - fromZone| + 1

public int calculateZonesCrossed() {
    return Math.abs(toZone - fromZone) + 1;
}


 // Gets discount rate from CityRideDataset.

public BigDecimal getDiscountRate() {
    return CityRideDataset.DISCOUNT_RATE.get(passengerType);
}


 // Gets daily cap from CityRideDataset.

public BigDecimal getDailyCap() {
    return CityRideDataset.DAILY_CAP.get(passengerType);
}
```

```
  // Getters and Setters
  public int getJourneyId() { return journeyId; }
  public void setJourneyId(int journeyId) { this.journeyId = journeyId; }

  public LocalDate getDate() { return date; }
  public void setDate(LocalDate date) { this.date = date; }

  public CityRideDataset.TimeBand getTimeBand() { return timeBand; }
  public void setTimeBand(CityRideDataset.TimeBand timeBand) { this.timeBand = timeBand; }

  public int getFromZone() { return fromZone; }
  public void setFromZone(int fromZone) { this.fromZone = fromZone; }

  public int getToZone() { return toZone; }
  public void setToZone(int toZone) { this.toZone = toZone; }

  public CityRideDataset.PassengerType getPassengerType() { return passengerType; }
  public void setPassengerType(CityRideDataset.PassengerType passengerType) { this.passenge

  public int getZonesCrossed() { return zonesCrossed; }
  public void setZonesCrossed(int zonesCrossed) { this.zonesCrossed = zonesCrossed; }

  public BigDecimal getBaseFare() { return baseFare; }
  public void setBaseFare(BigDecimal baseFare) { this.baseFare = baseFare; }

  public BigDecimal getDiscountApplied() { return discountApplied; }
  public void setDiscountApplied(BigDecimal discountApplied) { this.discountApplied = disco

  public BigDecimal getChargedFare() { return chargedFare; }
  public void setChargedFare(BigDecimal chargedFare) { this.chargedFare = chargedFare; }


   // Returns formatted string representation of the journey.

  public String toString() {
      DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
      return String.format("ID: %d | Date: %s | From: %d | To: %d | Time: %s | " +
                      "Passenger: %s | Zones: %d | Base: £%.2f | Charged: £%.2f",
              journeyId, date.format(formatter), fromZone, toZone, timeBand,
              passengerType, zonesCrossed, baseFare, chargedFare);
  }
```

}

journeyManager.java

import java.math.BigDecimal;

```
import java.util.ArrayList;

import java.util.HashMap;

import java.util.List;

import java.util.Map;

public class JourneyManager {
```

```
private List<Journey> journeys;
private int nextJourneyId;
private Map<CityRideDataset.PassengerType, BigDecimal> runningTotals;
private Map<CityRideDataset.PassengerType, Boolean> capReached;

public JourneyManager() {
    this.journeys = new ArrayList<>();
    this.nextJourneyId = 1;
    this.runningTotals = new HashMap<>(); // A HashMap stores data as key-value pairs, wh
    this.capReached = new HashMap<>();

    for (CityRideDataset.PassengerType type : CityRideDataset.PassengerType.values()) {
        runningTotals.put(type, BigDecimal.ZERO);
        capReached.put(type, false);
    }
}


 // Adds a journey with full fare calculation.
public Journey addJourney(Journey journey) {
    CityRideDataset.PassengerType type = journey.getPassengerType();
    BigDecimal currentTotal = runningTotals.get(type);
    boolean capAlreadyReached = capReached.get(type);

    BigDecimal chargedFare;

    if (capAlreadyReached) {
        chargedFare = BigDecimal.ZERO;
        journey.setChargedFare(chargedFare);
        journey.setZonesCrossed(journey.calculateZonesCrossed());
        journey.setBaseFare(BigDecimal.ZERO);
    } else {
        chargedFare = FareCalculator.calculateChargedFare(journey, currentTotal);
    }

    journey.setJourneyId(nextJourneyId);
    nextJourneyId++;

    // Update running total
    BigDecimal newTotal = runningTotals.get(type).add(chargedFare);
    runningTotals.put(type, newTotal);

    // Check if cap reached
    if (newTotal.compareTo(journey.getDailyCap()) >= 0) {
        capReached.put(type, true);
    }

    journeys.add(journey);

    return journey;
```

```
}


 // Gets all journeys.

public List<Journey> getAllJourneys() {
    return new ArrayList<>(journeys);
}


 // Gets journey by ID.

public Journey getJourneyById(int id) {
    for (Journey journey : journeys) {
        if (journey.getJourneyId() == id) {
            return journey;
        }
    }
    return null;
}

  //Gets journey count.

public int getJourneyCount() {
    return journeys.size();
}


 //Gets running total for a passenger type.

public BigDecimal getPassengerTotal(CityRideDataset.PassengerType type) {
    return runningTotals.get(type);
}


 // Checks if cap reached.

public boolean isCapReached(CityRideDataset.PassengerType type) {
    return capReached.get(type);
}


  //Gets total cost across all types.

public BigDecimal getTotalCost() {
    BigDecimal total = BigDecimal.ZERO;
    for (BigDecimal amount : runningTotals.values()) {
        total = total.add(amount);
    }
    return total;
}
```

```
   // Resets all data.

  public void resetDay() {
      journeys.clear();
      nextJourneyId = 1;
      for (CityRideDataset.PassengerType type : CityRideDataset.PassengerType.values()) {
          runningTotals.put(type, BigDecimal.ZERO);
          capReached.put(type, false);
      }
  }
```

}

InputValidator.java

import java.time.LocalDate;

import java.time.format.DateTimeFormatter;

import java.time.format.DateTimeParseException;

public class InputValidator {

```
private static final DateTimeFormatter DATE_FORMATTER = DateTimeFormatter.ofPattern("d/M/

public static boolean isValidZone(int zone) {
    return zone >= CityRideDataset.MIN_ZONE && zone <= CityRideDataset.MAX_ZONE;
}

public static boolean isValidDate(String dateStr) {
    // try-catch is used to handle runtime errors so the program does not crash
    try {
        LocalDate.parse(dateStr, DATE_FORMATTER);
        return true;
    } catch (DateTimeParseException e) {
        return false;
    }
}

public static LocalDate parseDate(String dateStr) {
    try {
        return LocalDate.parse(dateStr, DATE_FORMATTER);
    } catch (DateTimeParseException e) {
        return null;
    }
}

public static boolean isValidInteger(String input) {
    if (input == null || input.trim().isEmpty()) {
        return false;
    }
    try {
        Integer.parseInt(input.trim()); // trim() removes any leading and trailing whites
        return true;
    } catch (NumberFormatException e) {
        return false;
    }
}

public static boolean isValidTimeBand(String input) {
    if (input == null) return false;
    String upper = input.trim().toUpperCase();
    return upper.equals("PEAK") || upper.equals("OFF_PEAK");
}

public static CityRideDataset.TimeBand parseTimeBand(String input) {
    String upper = input.trim().toUpperCase();
    if (upper.equals("PEAK")) return CityRideDataset.TimeBand.PEAK;
    if (upper.equals("OFF_PEAK")) return CityRideDataset.TimeBand.OFF_PEAK;
    return null;
}

public static boolean isValidPassengerType(String input) {
```

```
    if (input == null) return false;
    String upper = input.trim().toUpperCase();
    return upper.equals("ADULT") || upper.equals("STUDENT") ||
            upper.equals("CHILD") || upper.equals("SENIOR_CITIZEN");
}

public static CityRideDataset.PassengerType parsePassengerType(String input) {
    String upper = input.trim().toUpperCase();
    switch (upper) {
        case "ADULT":
            return CityRideDataset.PassengerType.ADULT;
        case "STUDENT":
            return CityRideDataset.PassengerType.STUDENT;
        case "CHILD":
            return CityRideDataset.PassengerType.CHILD;
        case "SENIOR_CITIZEN":
            return CityRideDataset.PassengerType.SENIOR_CITIZEN;
        default: return null;
    }
}
```

}

FareCalculator.java

import java.math.BigDecimal;

import java.math.RoundingMode;

public class FareCalculator { // Calculates complete fare for a journey (base fare + discount + cap).

```
public static BigDecimal calculateChargedFare(Journey journey, BigDecimal currentTotal) {
    // Get base fare from dataset
    BigDecimal baseFare = CityRideDataset.getBaseFare(
            journey.getFromZone(), journey.getToZone(), journey.getTimeBand()
    );

    if (baseFare == null) {
        baseFare = BigDecimal.ZERO;
    }
    journey.setBaseFare(baseFare);

    // Calculate zones crossed
    int zonesCrossed = journey.calculateZonesCrossed();
    journey.setZonesCrossed(zonesCrossed);

    // Apply discount
    BigDecimal discountRate = journey.getDiscountRate();
    BigDecimal discountedFare = baseFare.multiply(BigDecimal.ONE.subtract(discountRate));
    discountedFare = discountedFare.setScale(2, RoundingMode.HALF_UP);

    // Apply daily cap
    BigDecimal dailyCap = journey.getDailyCap();
    BigDecimal chargedFare;

    if (currentTotal.compareTo(dailyCap) >= 0) {
        chargedFare = BigDecimal.ZERO;
    } else {
        BigDecimal newTotal = currentTotal.add(discountedFare);
        if (newTotal.compareTo(dailyCap) > 0) {
            chargedFare = dailyCap.subtract(currentTotal);
        } else {
            chargedFare = discountedFare;
        }
    }

    journey.setDiscountApplied(discountRate);
    journey.setChargedFare(chargedFare);

    return chargedFare;
}
```

}

MenuHandler.java

import java.math.BigDecimal; import java.time.LocalDate; import java.util.List; import java.util.Scanner;

```
public class MenuHandler {
```

```
private static final int MENU_ADD_JOURNEY = 1;
private static final int MENU_LIST_JOURNEYS = 2;
private static final int MENU_EXIT = 3;

private Scanner scanner;
private JourneyManager journeyManager;
private boolean running;

public MenuHandler() {
    this.scanner = new Scanner(System.in);
    this.journeyManager = new JourneyManager();
    this.running = true;
}

public void start() {
    displayWelcomeMessage();

    while (running) {
        displayMainMenu();
        int choice = getIntInput("Enter your choice: ");
        processMenuChoice(choice);
    }

    System.out.println("\nThank you for using CityRide Lite!");
    scanner.close(); //  used to close the Scanner object when you are finished using it
}

private void displayWelcomeMessage() {
    System.out.println("\n========================================");
    System.out.println("     WELCOME TO CITYRIDE LITE");
    System.out.println("========================================");
}

private void displayMainMenu() {
    System.out.println("\n========== MAIN MENU ==========");
    System.out.println("1. Add Journey");
    System.out.println("2. List All Journeys");
    System.out.println("3. Exit");
    System.out.println("================================");
}

private void processMenuChoice(int choice) {
    if (choice == MENU_ADD_JOURNEY) {
        addJourney();
    } else if (choice == MENU_LIST_JOURNEYS) {
        listAllJourneys();
    } else if (choice == MENU_EXIT) {
        running = false;
    } else {
        System.out.println("Invalid choice! Please enter 1-3.");
```

```
    }
}

private void addJourney() {
    System.out.println("\n--- Add New Journey ---");

    LocalDate date = getDateInput("Enter date (DD/MM/YYYY) or press Enter for today: ");
    CityRideDataset.TimeBand timeBand = getTimeBandInput();
    int fromZone = getZoneInput("Enter starting zone (1-5): ");
    int toZone = getZoneInput("Enter destination zone (1-5): ");
    CityRideDataset.PassengerType passengerType = getPassengerTypeInput();

    Journey journey = new Journey(0, date, timeBand, fromZone, toZone, passengerType);
    Journey completedJourney = journeyManager.addJourney(journey);

    System.out.println("\n✓ Journey added successfully!");
    System.out.println("  Journey ID: " + completedJourney.getJourneyId());
    System.out.printf("  Zones crossed: %d%n", completedJourney.getZonesCrossed());
    System.out.printf("  Base fare: £%.2f%n", completedJourney.getBaseFare());
    System.out.printf("  Charged fare: £%.2f%n", completedJourney.getChargedFare());

    if (journeyManager.isCapReached(passengerType)) {
        System.out.println("  ⚠ Daily cap reached for " + passengerType + "!");
    }
}

private void listAllJourneys() {
    List<Journey> journeys = journeyManager.getAllJourneys();

    System.out.println("\n--- All Journeys ---");

    if (journeys.isEmpty()) {
        System.out.println("No journeys recorded yet.");
    } else {
        System.out.println("\nTotal journeys: " + journeys.size());
        System.out.println("-".repeat(80));

        for (Journey journey : journeys) {
            System.out.println(journey);
        }

        System.out.println("-".repeat(80));
        System.out.printf("Total cost for the day: £%.2f%n", journeyManager.getTotalCost(
    }
}

private LocalDate getDateInput(String prompt) {
    boolean valid = false;
    String input = "";

    while (valid == false) {
```

```
        System.out.print(prompt);
        input = scanner.nextLine().trim();

        if (input.isEmpty()) {
            return LocalDate.now();
        }

        if (InputValidator.isValidDate(input)) {
            valid = true;
        } else {
            System.out.println("Error: Invalid date format! Use DD/MM/YYYY");
        }
    }

    return InputValidator.parseDate(input);
}

private CityRideDataset.TimeBand getTimeBandInput() {
    boolean valid = false;
    CityRideDataset.TimeBand band = null;

    while (valid == false) {
        System.out.print("Enter time band (PEAK / OFF_PEAK): ");
        String input = scanner.nextLine().trim().toUpperCase();

        if (InputValidator.isValidTimeBand(input)) {
            band = InputValidator.parseTimeBand(input);
            valid = true;
        } else {
            System.out.println("Error: Invalid time band! Enter PEAK or OFF_PEAK");
        }
    }
    return band;
}


private int getZoneInput(String prompt) {
    boolean valid = false;
    int zone = -1;

    while (valid == false) {
        zone = getIntInput(prompt);

        if (InputValidator.isValidZone(zone)) {
            valid = true;
        } else {
            System.out.println("Error: Zone must be between 1 and 5");
        }
    }

    return zone;
```

```
}

private CityRideDataset.PassengerType getPassengerTypeInput() {
    boolean valid = false;
    CityRideDataset.PassengerType type = null;

    while (!valid) {
        System.out.print("Enter passenger type (ADULT/STUDENT/CHILD/SENIOR_CITIZEN): ");
        String input = scanner.nextLine().trim().toUpperCase();

        if (InputValidator.isValidPassengerType(input)) {
            type = InputValidator.parsePassengerType(input);
            valid = true;
        } else {
            System.out.println("Error: Invalid passenger type!");
        }
    }

    return type;
}

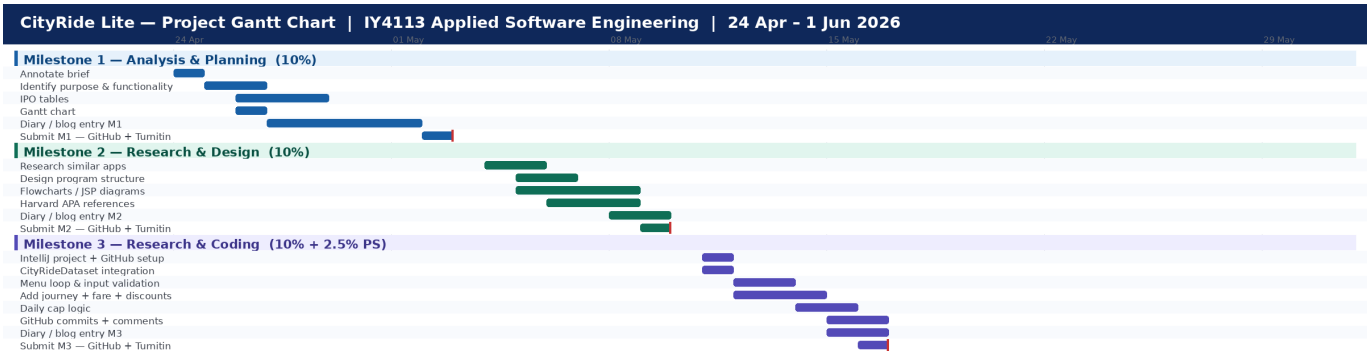

private int getIntInput(String prompt) {
    boolean valid = false;
    int value = 0;

    while (!valid) {
        System.out.print(prompt);
        String input = scanner.nextLine().trim();

        if (input.isEmpty()) {
            System.out.println("Error: Input cannot be blank!");
        } else if (InputValidator.isValidInteger(input)) {
            value = Integer.parseInt(input);
            valid = true;
        } else {
            System.out.println("Error: Please enter a valid number!");
        }
    }

    return value;
}
```

Updated Gantt Chart



Diary Entries

Entry 1 – Date [10/05/2026] Research on OOP concepts

During this stage of the project, I focused on researching programming techniques and software design principles to help improve the quality and structure of the CityRide Lite system.

I researched object-oriented programming concepts using GeeksforGeeks and Oracle Java documentation. This helped me understand how to organise the application into multiple classes, each with its own job.

I also researched Java collections, especially ArrayList, because the system needs to store journeys while the program is running. This research helped me understand how collections can be used to add, remove, and show journey objects efficiently.

Another important area of research was financial calculations in Java. I looked into how to handle money values correctly and learned that BigDecimal is more accurate than double for currency calculations. I also researched input validation and exception handling techniques to reduce the risk of errors caused by bad user input while the program is running.

One problem I faced was knowing when to use BigDecimal instead of double, because both can store decimal numbers. I also had trouble at first understanding how enums work in Java and how they help with validation and making code easier to read.

Another challenge was deciding how to organise the classes so that each class had its own job, without making the design too complicated or adding unnecessary part

Entry 2 – Date [11/05/2026] Input Validation

Today I implemented the InputValidator and MenuHandler classes. The InputValidator class includes methods to validate all user inputs. This includes checking that zones are numeric values between 1 and 5, passenger types match one of the four allowed values using case-

insensitive comparison, time bands are either “peak” or “off-peak”, dates follow the correct format, and journey IDs exist before a journey can be removed.

The MenuHandler class is responsible for displaying the main menu and sub-menus, collecting user input, and using the InputValidator to keep prompting the user until valid data is entered. I added validation loops so the program repeatedly asks for input and displays clear error messages whenever invalid data is entered. I also separated the validation logic from the menu handling logic so that each class has a single responsibility, making the code more organised and easier to maintain.

One challenge I faced was handling non-numeric input for zones. If a user enters letters instead of numbers, Integer.parseInt throws a NumberFormatException. To solve this, I used a try-catch block and prompted the user again until a valid number was entered. Another issue was making passenger type input case-insensitive. I solved this by converting the user’s input to uppercase before comparing it with the enum names. I also considered allowing abbreviations, but decided to only accept full names to keep the validation simple and avoid confusion.

Entry 3 – Date [13/05/2026] Creating the JourneyManager Class

Today I implemented the JourneyManager class, which acts as the main service for handling all journey storage and operations. The class uses an ArrayList to store journeys in memory and generates unique session IDs for each new journey using an auto-incrementing counter. I added methods for adding journeys, removing journeys by ID with a confirmation step, and resetting all journeys after user confirmation.

I also implemented filtering features that allow users to search for journeys by passenger type, time band, zone, and date. The zone filtering checks both the starting zone and destination zone to find matching journeys. In addition, I added logic to maintain separate running totals for each passenger type, which is necessary for correctly applying daily fare caps. I created this class to centralise all journey-related storage and processing, making the main program cleaner, more organised, and easier to maintain.

One of the main challenges was designing the zone filtering logic. I needed to decide whether a journey should match only when the zone is the starting zone, the destination zone, or either one. I chose to use “or” logic, meaning a journey is included if the specified zone matches either the from zone or the to zone, as this would be more useful for users wanting to view all journeys connected to a particular zone.

Another challenge was updating the running totals when a journey is removed. Instead of adjusting the totals manually after each deletion, I created a recalculateTotals method that loops

through all remaining journeys and rebuilds the totals from the beginning. This approach is simpler, easier to maintain, and reduces the risk of calculation errors.

Entry 4 – Date [14/05/2026] Creating the FareCalculator class

Today I implemented the FareCalculator class, which handles all fare calculations in the program. I researched how to access the fare data from the provided CityRideDataset.java class so I could understand how the dataset was structured. After understanding it, I created three main methods.

The first method calculates the number of zones crossed.

$|toZone - fromZone| + 1$

The second method looks up the base fare using the from zone, to zone, and time band. The third method applies the passenger discount using the values stored in the PassengerType enum.

I also started implementing the cap logic, which makes sure the total fare does not go above the daily cap. I separated these calculations into their own class to keep the code organised and easier to manage.

The main problem I faced was understanding the structure of the dataset and finding the correct lookup methods. Another challenge was implementing the cap logic correctly so that if the total exceeds the cap, only the remaining amount up to the cap is charged instead of the full fare.

Entry 5 – Date [15/05/2026] Finalising the classes and making

Today I created the Main class to connect all parts of the program into a working application. The Main class contains the main program loop, which displays the menu and keeps the program running until the user chooses to exit.

I connected all the classes I previously created, including the MenuHandler for user input, the JourneyManager for storing and managing journeys, the FareCalculator for fare calculations, and the ConsolePrinter for displaying output.

I implemented all the menu options. The add journey option collects user details, validates the inputs, calculates the fare, applies the daily cap logic, and stores the journey. The list journeys option displays all journeys entered by the user. The filter journeys option allows users to filter journeys by passenger type, time band, zone, or date. The daily summary and totals by passenger type options display the required information using the ConsolePrinter methods. The remove journey and reset day options both ask for confirmation before making changes.

After finishing the implementation, I carried out basic testing on each feature to check that the program was working correctly.

References

GeeksforGeeks (2024).*Object oriented programming in Java*. Available at:Java OOP(Object Oriented Programming) Concepts - GeeksforGeeks(Accessed: 11 May 2026).

Oracle (2025).*The Collections Framework*. Available at:Trail: Collections (The Java™ Tutorials) (Accessed: 11 May 2026).

TutorialsPoint (2024).*Java Exception Handling*. Available at:Java - Exceptions(Accessed: 11 May 2026).